

Meccanismi di sincronizzazione

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 11

Indice

1	Introduzione	2
2	Definizione del problema	2
3	Metodologia	3
3.1	Esercizio 1 - Assenza di sincronizzazione	3
3.2	Esercizio 2 - Mutua esclusione con mutex	3
3.3	Esercizio 3 - Controllo concorrente con semaforo	3
4	Risultati sperimentali	4
4.1	Assenza di sincronizzazione	4
4.2	Utilizzo del mutex	4
4.3	Utilizzo del semaforo	4
5	Confronto	5
6	Conclusioni	5

1 Introduzione

Nel presente laboratorio vengono analizzati i principali problemi legati alla concorrenza tra thread, con particolare attenzione ai meccanismi di sincronizzazione impiegati per garantire la correttezza nell'accesso a risorse condivise.

In ambienti concorrenti, più thread possono accedere simultaneamente a una stessa risorsa, rendendo necessario l'utilizzo di opportuni strumenti per evitare inconsistenze nei dati.

In particolare, si analizza il fenomeno delle *race condition*, ovvero situazioni in cui il risultato di un'esecuzione dipende dall'ordine temporale non deterministico delle operazioni.

A tal fine, sono stati sviluppati tre programmi distinti:

- un programma privo di meccanismi di sincronizzazione;
- un programma che utilizza un mutex per garantire la mutua esclusione;
- un programma che utilizza un semaforo per limitare l'accesso concorrente.

L'obiettivo è osservare e confrontare il comportamento dei tre approcci, evidenziando il compromesso tra correttezza e parallelismo.

2 Definizione del problema

Il laboratorio considera due problemi legati alla programmazione concorrente.

Il primo problema riguarda l'accesso concorrente a una variabile condivisa, denominata **counter**, da parte di più thread. Questo caso viene analizzato sia in assenza di sincronizzazione sia mediante l'utilizzo di un mutex. Ogni thread esegue un numero fissato di incrementi, pari a:

$$MAX_VALUE = 10000$$

Poiché vengono creati 10 thread, il valore finale atteso della variabile è:

$$counter = NUM_THREADS \times MAX_VALUE = 10 \times 10000 = 100000$$

L'operazione di incremento non è atomica, in quanto può essere scomposta nelle seguenti operazioni:

```
temp = counter;  
temp++;  
counter = temp;
```

Se questa sequenza viene eseguita contemporaneamente da più thread senza alcun meccanismo di sincronizzazione, possono verificarsi interferenze tra le operazioni e perdita di aggiornamenti.

Il secondo problema riguarda invece il controllo dell'accesso concorrente a una sezione critica. In questo caso viene realizzato un nuovo programma con 5 thread identici, nel quale un semaforo limita a 2 il numero massimo di thread che possono trovarsi contemporaneamente nella sezione critica.

3 Metodologia

3.1 Esercizio 1 - Assenza di sincronizzazione

Nel primo programma non viene utilizzato alcun meccanismo di sincronizzazione. 10 thread accedono simultaneamente alla variabile condivisa.

Ogni thread esegue le seguenti operazioni:

- lettura del valore corrente della variabile;
- incremento del valore;
- scrittura del nuovo valore.

L'assenza di coordinamento tra i thread introduce una condizione di competizione (*race condition*), poiché più thread possono accedere contemporaneamente alla stessa risorsa.

3.2 Esercizio 2 - Mutua esclusione con mutex

Nel secondo programma viene introdotto un mutex per proteggere la sezione critica. L'accesso alla variabile condivisa è racchiuso tra le seguenti operazioni:

```
pthread_mutex_lock(&mutex);  
...  
pthread_mutex_unlock(&mutex);
```

Questo meccanismo garantisce la mutua esclusione, consentendo a un solo thread alla volta di accedere alla sezione critica ed eliminando le condizioni di competizione.

3.3 Esercizio 3 - Controllo concorrente con semaforo

Nel terzo programma viene utilizzato un semaforo contatore per limitare il numero di thread che possono accedere contemporaneamente alla sezione critica. A differenza dei primi due esercizi, questo programma è indipendente dal codice precedente e lancia 5 thread identici.

Il semaforo è inizializzato con valore pari a 2:

```
sem_init(&access_limiter, 0, 2);
```

In questo modo, al massimo due thread possono trovarsi contemporaneamente nella sezione critica. Ogni thread richiede l'accesso tramite:

```
sem_wait(&access_limiter);
```

e lo rilascia al termine della sezione critica tramite:

```
sem_post(&access_limiter);
```

Questo approccio non realizza una mutua esclusione stretta come il mutex, ma consente un accesso concorrente controllato, imponendo un limite massimo al numero di thread attivi nella sezione critica.

4 Risultati sperimentali

4.1 Assenza di sincronizzazione

Il comportamento osservato è non deterministico. In assenza di sincronizzazione, più thread possono accedere contemporaneamente alla variabile condivisa, leggendo e scrivendo il valore del `counter` senza alcun coordinamento.

Dall'analisi dell'output si può osservare che:

- più thread possono leggere lo stesso valore della variabile condivisa;
- il valore finale può risultare inferiore a quello atteso;
- alcuni incrementi possono essere persi.

Questi effetti sono riconducibili alla presenza di race condition. Tuttavia, poiché il comportamento dipende dall'ordine di esecuzione dei thread, non è garantito che l'errore si manifesti in ogni singola esecuzione.

4.2 Utilizzo del mutex

L'introduzione del mutex consente di proteggere l'accesso alla variabile condivisa. In particolare:

- il valore finale della variabile corrisponde a quello atteso;
- non si verificano perdite di aggiornamenti;
- gli accessi alla sezione critica sono serializzati.

Il mutex garantisce quindi la correttezza del valore finale del contatore, poiché un solo thread alla volta può modificare la variabile condivisa. L'ordine di esecuzione dei thread rimane comunque non deterministico, in quanto dipende dalla pianificazione del sistema operativo.

4.3 Utilizzo del semaforo

L'utilizzo del semaforo evidenzia un comportamento coerente con l'obiettivo dell'esercizio. Si osserva che:

- al massimo due thread accedono contemporaneamente alla sezione critica;
- gli altri thread rimangono in attesa finché uno dei thread già entrati non rilascia il semaforo;
- il livello di parallelismo è maggiore rispetto al caso del mutex, poiché non viene imposto l'accesso esclusivo di un solo thread alla volta.

La presenza di due thread contemporaneamente nella sezione critica non rappresenta un errore, ma il comportamento richiesto dal programma. Il semaforo, infatti, non viene utilizzato per eliminare completamente la concorrenza, ma per limitarla a un numero massimo di accessi simultanei.

5 Confronto

Approccio	Garanzia fornita	Parallelismo	Complessità
No Sync	Non fornisce garanzie sulla correttezza del risultato, poiché l'accesso al contatore condiviso non è sincronizzato.	Parallelismo alto.	Bassa.
Mutex	Garantisce che un solo thread alla volta acceda alla sezione critica, evitando perdite di aggiornamenti.	Parallelismo basso nella sezione critica.	Media.
Semaforo	Limita l'accesso concorrente consentendo al massimo due thread contemporaneamente nella sezione critica.	Parallelismo medio.	Media.

Tabella 1: Confronto tra i meccanismi di sincronizzazione

I risultati evidenziano che i diversi meccanismi di sincronizzazione rispondono a esigenze differenti. L'assenza di sincronizzazione permette un elevato livello di parallelismo, ma non garantisce la correttezza del risultato.

Il mutex è adatto quando è necessario garantire la mutua esclusione, impedendo che più thread accedano contemporaneamente alla stessa risorsa critica.

Il semaforo, invece, è utile quando si vuole controllare il numero massimo di thread ammessi contemporaneamente in una sezione critica.

La scelta del meccanismo più appropriato dipende quindi dall'obiettivo del programma: garantire la correttezza di una variabile condivisa, serializzare l'accesso a una risorsa oppure limitare il grado massimo di concorrenza.

6 Conclusioni

Il laboratorio ha consentito di analizzare sperimentalmente i problemi legati alla concorrenza tra thread e l'importanza dei meccanismi di sincronizzazione.

L'assenza di sincronizzazione ha evidenziato chiaramente il problema delle race condition, mostrando comportamenti non deterministici e risultati errati.

L'introduzione del mutex ha garantito la mutua esclusione, assicurando la correttezza dell'esecuzione al costo di una riduzione del parallelismo.

L'utilizzo del semaforo ha invece mostrato un approccio intermedio, in grado di bilanciare parallelismo e controllo dell'accesso, pur senza garantire completamente la sicurezza delle operazioni.

In conclusione, la gestione efficace della concorrenza richiede una scelta consapevole dei meccanismi di sincronizzazione, in funzione degli obiettivi del sistema, quali correttezza, efficienza e scalabilità.